Inicio Java

FUNDAMENTOS PARA EMPEZAR A APRENDER JAVA

Empezamos con Java

Es un lenguaje de programación orientado a objetos (POO)

Características:

- Tipos primitivos, enumerados y constantes
- Colecciones
- Expresiones regulares (Regex)
- POO en Java

Tipos primitivos

Tipo	Tamaño	Rango
byte	1 byte	Números enteros de -128 a 127
short	2 bytes	Números enteros de -32,768 a 32,767
int	4 bytes	Números enteros de -2,147,483,648 a 2,147,483,647
long	8 bytes	Números enteros de -9,223,372,036,854,775,808 a 9,223,372,036,854,775,807
float	4 bytes	Números decimales con 6 a 7 digitos decimales
double	8 bytes	Números decimales con 15 dígitos decimales
char	2 bytes	Almacena un caracter o codigo ASCII
boolean	1 byte	Almacena valores verdaderos o falsos (true y false)

Tipos primitivos

Existen en java clases equivalentes a los tipos primitivos, las cuales disponen de un abanico de funcionalidad que permite, entre otras muchas cosas, realizar la conversión de tipos de manera adecuada. Estas clases tienen el mismo nombre que el tipo primitivo, pero en notación Pascal (**Integer, Character**, etc).

```
Integer integer = 300;
String string = integer.toString();

int suma = integer+50;
String concat = string+50;

System.out.println("integer = "+integer);
System.out.println("string = "+string);
System.out.println("suma = "+suma);          // resultado?
System.out.println("concat = "+concat);      //resultado?
```

Cadenas

Se utiliza la clase **String** para trabajar con cadenas de caracteres.

Se pueden comparar, concatenar y trabajar con cadenas de caracteres gracias a esta clase y alguna optimización de la misma, como puede ser la **StringBuffer**:

```
String opcion = "salir";
if(opcion=="salir")                              //comparación
    System.out.println("lo ha comparado?");

String nombre = "Cristina";
String apellidos = "Pérez Grau";
String nombreCompleto = nombre+" "+apellidos;    //concatenación
System.out.println("Nombre: "+nombreCompleto);

StringBuffer sB = new StringBuffer();            //usamos clase optimizada
sB.append(nombre);
sB.append(" ");
sB.append(apellidos);
System.out.println("sB = "+sB);
```

Constantes

En Java las constantes se declaran como valores **estáticos (static) y finales (final).**

- **Valor (o método) estático**, el valor (o método) estará disponible incluso aunque no tengamos una instancia u objeto de la clase que lo ha declarado.

  Ejemplo de método estático de la clase Operación:

  Operación.**sumar**(op1, op2) //¿Cómo debería ser declarado en la clase?

- **Valor final,** indica que no se puede modificar. Por tanto, una declaración de constante sería algo así:

  static final int MAX VIDAS = 3;

  ¿Ejemplos claros de valores estáticos?

Ejercicio 1

Implementar la clase llamada Operacion. Definir dos métodos estáticos que permitan sumar y restar dos valores enteros y también definir un valor constante comprobando que no se va a poder modificar fuera de esta la clase.

Implementar un programa que muestre por pantalla el resultado de dos operaciones, una suma y una resta para dos números dados, estas operaciones se definen como métodos dentro de la clase “Operación” y para realizarlas no será necesario instanciar un objeto puesto que son métodos estáticos.

Intenta modificar la constante que has creado dentro de la clase ¿se puede? ¿por qué?

Enumerados

Son rangos integrados por conjunto de valores constantes. Ejemplo: los **nombres de los días semana o de los meses.**

En Java, los valores enumerados son clases. Esto quiere decir que, al igual que el resto de clases, **pueden tener atributos y métodos** (incluido un constructor).

La única **diferencia** es que los valores definidos en el enumerado son **public, static y final** (y no se puede modificar).

NO se pueden instanciar objetos de un enumerado **ni pueden heredar** de otra clase.

Ejemplo de enumerados

```
enum Dias {  4 usages
    LUNES, MARTES, MIERCLES, JUEVES, VIERNES, SABADO, DOMINGO;  1 usage

    static boolean finDeSemana(Dias dia) {  1 usage
        return (dia == SABADO || dia == DOMINGO);
    }
}
```

```
Dias miDia = Dias.DOMINGO;
if (Dias.finDeSemana( miDia ) ) {
    System.out.println("El día es fin de seana");
}
else
    System.out.println("El día no es fin de semana");
```

Colecciones

Uno de los **principales problemas** cuando desarrollamos una aplicación informática es el **tratamiento (manejo) de los datos.**

En el desarrollo de cualquier aplicación resulta **imprescindible** poder disponer de **estructuras de datos que nos faciliten el manejo básico de los mismos**.

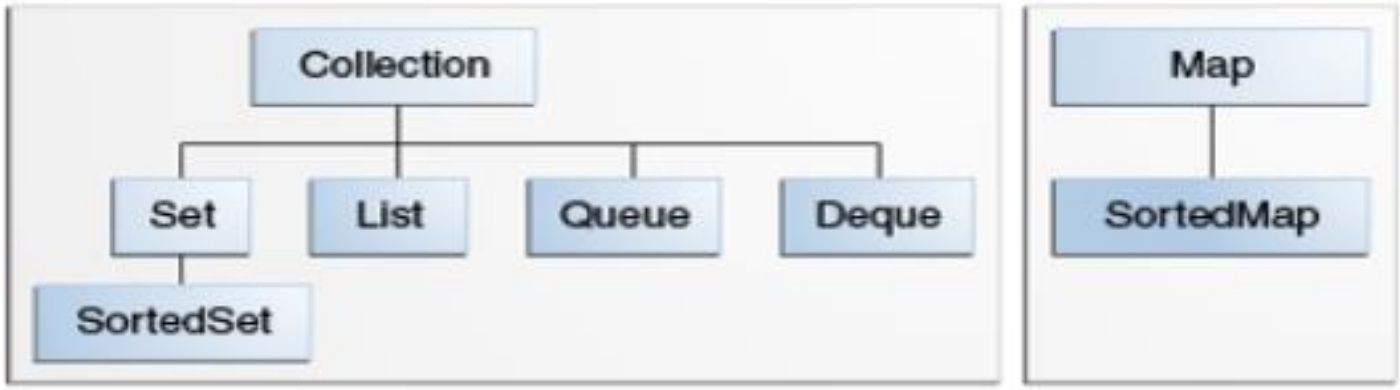
Con este objetivo, en Java disponemos de la **interfaz Collection** (colección): Una colección es un grupo de objetos conocidos como elementos.

Las colecciones proporcionan formas de almacenar, recuperar y manipular datos de manera eficiente. Java proporciona una serie de clases e interfaces para trabajar con colecciones, cada una de ellas diseñada para satisfacer diferentes necesidades de programación.

Colecciones

Los tipos principales de colecciones en Java se pueden dividir en cuatro categorías:

- Listas
- Mapas
- Conjuntos
- Colas



Vamos a ver un pequeño resumen de lo que es cada una.

Map, se considera una interfaz de colección principal de Java Collections Framework.

Documentación oficial Collections Java

Listas

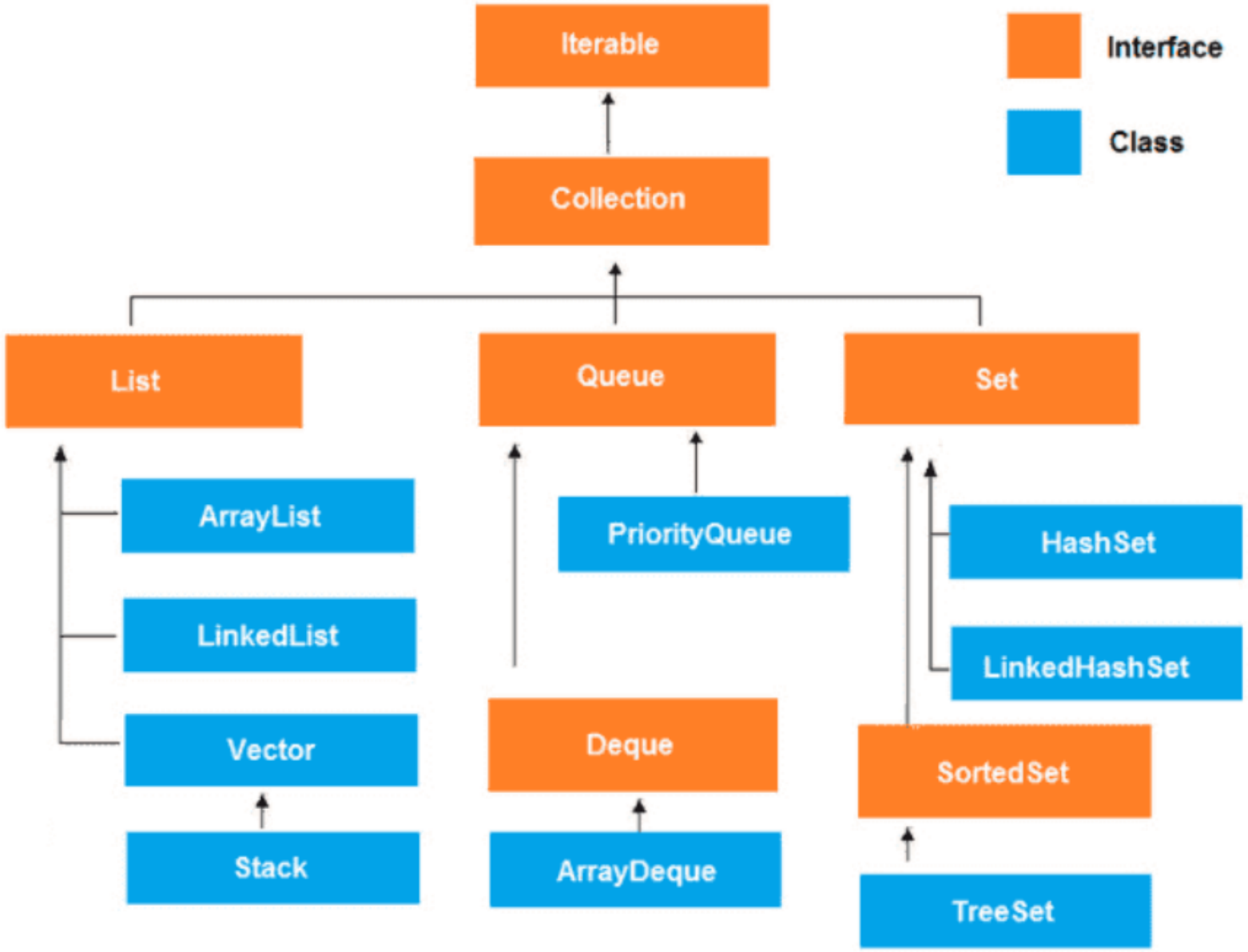
La interfaz List<> **permiten almacenar una secuencia ordenada de elementos**, donde cada elemento tiene un **índice** asociado.

Las listas son útiles cuando se necesita **acceder a los elementos por su posición** (índice), **o** cuando puede haber **elementos repetidos**.

La interfaz List<> **implementa la interfaz Iterable<>,** esto quiere decir que **se pueden usar los métodos de dicha interfaz para recorrer las listas**.

4 implementaciones diferentes:

- ArrayList
- LinkedList
- Vector
- Stack


Iterable
Collection
Interface
Class
List
Queue
Set
ArrayList
LinkedList
Vector
Stack
PriorityQueue
Deque
ArrayDeque
HashSet
LinkedHashSet
SortedSet
TreeSet

Listas

- **ArrayList**: utiliza como estructura de datos subyacente un **array**.
  - Accesos a elementos(posiciones), muy ràpidos
  - No la mejor opción si queremos listas ordenadas o puede aumentar o disminuir con frecuencia (redimensionar el array)
- **LinkedList**: lista enlazada o doblemente enlazada.
  - Buena opción para mantener las listas ordenadas.
  - Optimiza el uso de la memoria.
  - Sin embargo, los accesos a elementos usando el índice no son óptimos.
- **Vector**: implementación muy similar a ArrayList, pero sincronizada. Útil en caso de que se esté usando acceso concurrente a la lista.
- **Stack** (pilas): es una implementación de una lista LIFO (last in, first out). Extiende la interfaz Vector.

Ejercicios listas

- **Ejercicio 2**: Crea una lista de todos los números del 1 a 100 que sean divisibles por un número que introduzca el usuario. Este número tiene que estar comprendido entre 2 y 7. Pinta la lista por pantalla.
- **Ejercicio 3:** Crea una lista de enteros que tenga 10 números. La lista estará hardcodeada con los números que quieras, o puedes investigar cómo insertar números aleatorios.

  El programa tiene que ordenar la lista y presentarla por pantalla. ¿Cómo ordenarías?

  Luego pide al usuario que introduzca un límite inferior y otro superior. Entonces el programa elimina todos los elementos de la lista que estén fuera de los límites, incluyendo estos. Presenta los resultados por pantalla.
- **Ejercicio 4**: Crea una lista enlazada de cadenas de caracteres y realiza las siguientes operaciones.
  - Inicialmente añade nombres (método add()), añade otro nombre en una posición en concreto(método add(index)), elimina el primer nombre(método remove(index)), elimina elemento que coincida con un nombre(método add(String)), muestra la cantidad de elementos que contiene la lista, comprueba si un nombre está en la lista, vacía la lista y comprueba si se ha vaciado
  - Realiza todas estas mismas operaciones con una lista de tipo ArrayList. ¿Has utilizado los mismos métodos? ¿Funcionan igual internamente estos métodos?

Mapas

La interfaz **Map<key, value>** almacena **pares clave-valor** únicos, donde cada clave está asociada a un valor.

No puede haber claves duplicadas, pero sí valores duplicados. Los mapas **optimizan las búsquedas por los valores de la clave**, y son **la solución más adecuada cuando cada elemento dispone de una clave única**, y se van a realizar un gran número de búsquedas.

A la hora de crear un mapa debemos de tener en cuenta que, para los valores de clave, deberemos de utilizar, en vez de los tipos elementales, las clases equivalentes:

Map<Integer, String> mapa = **new HashMap<Integer, String>** ();

Véase el API de Map par ver implementaciones de la interfaz Map.

Mapas

Algunos de los métodos más utilizados de estas clases:

```
Map<Integer, String> nombreMap = new HashMap<Integer, String>();
nombreMap.size(); // Devuelve el numero de elementos del Map
nombreMap.isEmpty(); // Devuelve true si no hay elementos en el Map y false si si los hay
System.out.println("Tamaño del map: "+nombreMap.size());
nombreMap.put(clave, valor); // Añade un elemento al Map
nombreMap.get(clave); // Devuelve el valor de la clave que se le pasa como parámetro o 'null' si la clave no existe
nombreMap.clear(); // Borra todos los componentes del Map
nombreMap.remove(clave); // Borra el par clave/valor de la clave que se le pasa como parámetro
nombreMap.containsKey(clave); // Devuelve true si en el map hay una clave que coincide con K
nombreMap.containsValue(valor); // Devuelve true si en el map hay un Valor que coincide con V
nombreMap.values(); // Devuelve una "Collection" con los valores del Map
```

(Ver ejemplo de Maps_Ej (HashMap, TreeMap, LinkedHashMap)

Ejercicio HashMap

1. En primer lugar, piensa en cómo **Crear un HashMap** donde la clave sea el **ISBN** (un número que identifica de manera única a los libros) y el valor sea el **nombre del libro**.
2. Crear un programa para gestionar el préstamo de libros de una biblioteca. Se debe permitir al usuario realizar las siguientes acciones:
    1. Añadir un nuevo libro.
    2. Eliminar un libro existente.
    3. Consultar si un libro está disponible dado un ISBN.
    4. Mostrar la lista completa de libros.
3. Usa un menú para que el usuario pueda elegir qué acción realizar.

- Nota: Si el usuario intenta agregar un libro con un ISBN que ya existe, el programa debe advertir que no se puede duplicar.

Conjuntos - Set

Es una colección **desordenada** de objetos en los que **no se pueden almacenar valores duplicados.**

Los conjuntos son **útiles cuando se necesita garantizar que no haya duplicados en los datos y cuando no es necesario mantener un orden específico**, simplemente verificar si el elemento pertenece o no al conjunto.

La **interfaz Set** define los **métodos básicos** de añadir, eliminar, comprobar tamaño, si está vacío, también podemos verificar si un elemento existe en el conjunto (Set). Pero **no hay forma de obtener un elemento del conjunto**.

- Se puede comprobar que algo existe y podemos iterar sobre los elementos del conjunto,
- pero no es posible obtener, por ejemplo, el elemento 3 del conjunto.

Conjuntos

Existen varias implementaciones, entre las cuales:

- java.util.HashSet
- java.util.LinkedHashSet
- java.util.TreeSet

```
Set setA = new HashSet();
Set setB = new LinkedHashSet();
Set setC = new TreeSet();
```

```
Set<Integer> set = new HashSet<Integer>();
```

RETO: Buscad en el API de Java los métodos que se pueden utilizar con conjuntos y practicadlos.

Cread una clase, por ejemplo “Persona” e insertar elementos de esa clase que sean iguales en vuesto conjunto ¿os deja? ¿por qué?

Expresiones regulares (Regex)

Una **de las operaciones que más realizamos** al escribir código es **comprobar el formato de los datos**: comprobar **código postal** (5 caracteres numéricos), **matrículas, emails**, etc...

Esta operación realizada sobre variables de tipo cadena puede ser bastante engorrosa, **se puede simplificar mediante el uso de expresiones regulares**.

Una **expresión regular** es una cadena, formada por una serie de caracteres especiales, que **definen el formato que deben seguir el conjunto de cadenas que se deben validar**

Expresiones regulares Regex

Caracteres regex	Función
[]	Cualquier carácter del conjunto será válido
()	Grupo de caracteres formado por uno o varios caracteres y que pueden operarse unos dentro de los otros.
-	Funciona a modo de especificación del área (de [...] hasta [...]) cuando se sitúa entre dos caracteres normales.
^	Limita la búsqueda al inicio de una línea. También permite negar una clase de caracteres, si se encuentra dentro de unos corchetes.
$	Limita la búsqueda al final de una línea.
|	Permite establecer alternativas (un resultado u otro)
.	Equivale a cualquier carácter.
*	El número del carácter, de la clase o del grupo situado antes de * es opcional (cero incluido).
+	El carácter, la clase o el grupo antes de + debe aparecer como mínimo una vez.
?	El carácter, la clase o el grupo antes de ? es opcional y puede aparecer como máximo una vez .
{n}	El carácter, la clase o el grupo anteriores aparecen exactamente n veces.

Expresiones regulares Regex

{n, m}	El carácter, la clase o el grupo anteriores aparecen como mínimo n veces y como máximo m veces.
{n, }	El carácter, la clase o el grupo anteriores aparecen como mínimo n veces o con frecuencia.
\b	Tiene en cuenta el límite de palabra durante la búsqueda.
\B	Ignora el límite de palabra durante la búsqueda.
\d	Cualquier dígito; abreviatura para la clase de caracteres [0-9].
\D	Cualquier no dígito; abreviatura para la clase de caracteres [^0-9].
\w	Cualquier carácter alfanumérico; abreviatura para la clase de caracteres [a-zA-Z_0-9].
\W	Cualquier carácter no alfanumérico; abreviatura para la clase de caracteres [^\w].

Expresiones regulares Regex

Ejemplos

- Para validar un código postal, podríamos usar las expresiones:

  [0-9] {5} o, de manera simplificada: ^\d{5}$

- Una matrícula de las nuevas (sin control de letras), podría hacerse así:

  ^\d{4} [A-Z]{3}

- Para no usar las vocales (tal y como se hace en realidad), la expresión podría ser

  ^\d{4} [B-D,F-H,J-N,P-T,V-Z]{3}$

Expresiones regulares Regex

- Si quisiéramos aceptar una matrícula antigua, en formato una o dos letras, 4 números y dos letras, o 5 y 6 números, sin letras, podría ser:

  [A-Z]{1,2} (\d{4} [A-Z]{1,2}|\d{4}\d{0,2})$

- Y podríamos aceptar matrículas antiguas y nuevas combinando las dos expresiones anteriores:

  ^\d{4} [B-D,F-H,J-N,P-T,V-Z]{3}$|[A-Z]{1,2} (\d{4} [A-Z]{1,2}|\d{4}\d{0,2})$

  Existen muchas páginas donde se puede aprender mucho más sobre expresiones regulares y también donde se puede comprobar online nuestras expresiones, como por ejemplo https://regex101.com/.

Expresiones regulares Regex

En Java, la verificación de una cadena mediante expresiones regulares se hace a través del paquete **java.util.regex**, y consiste básicamente en dos clases:

- La clase **Pattern**, que es una versión compilada de la expresión regular que queremos verificar.
- La clase **Matcher**, que básicamente verifica si nuestro Pattern se encuentra dentro de la cadena de búsqueda.

Ejemplo:

```
Pattern miPattern = Pattern.compile( regex: "abc" ) ;
String cadena = "abc";
Matcher miMatcher = miPattern.matcher(cadena);
if ( miMatcher.matches () ) {
    System.out.println("OK");
}
else{
    System.out.println("NO OK");
}
```

Ejercicio 5

Realiza un programa en java que dado un email que se introduce por pantalla, se verifique si una dicha dirección de email es válida, mediante las clases que tiene java para tratar las expresiones regulares (Regex)

- Una dirección de correo electrónico es válida si:
  - La primera parte puede contener cualquier número y letra y los sígnos “.”, “_”, y “-”
  - Seguido de “@”
  - La tercera parte (nombre servidor o dominio), puede contener cualquier número y letra y los sígnos “_” y “-”
  - Seguido de un punto y entre 2 y 4 letras
  - Esta última parte (extensión del dominio solo) podrá aparecer 4 veces como máximo. (fijáos en vuesto correo de la identidad digital)

POO

Clases en un fichero

- Una **clase** en Java es una plantilla o **estructura** que define las **propiedades** (atributos) y **comportamientos** (métodos) que tendrán sus objetos
  - **Definición de una clase:**
    - **Atributos**: Son las **variables** dentro de la clase que definen las características de los objetos (como el color o el tamaño).
    - **Métodos**: Son las **funciones** dentro de la clase que definen lo que los objetos pueden hacer (como caminar o hablar, si hablamos de una clase Persona).

```
public class Person {  no usages
    // Atributos
    private int id;  no usages
    private String nombre;  1 usage

    // Método: comportamiento
    public void saludar() {  no usages
        System.out.println("Hola, mi nombre es "
                + nombre);
    }
}
```

POO

En Java todos los elementos (clases, métodos, atributos, propiedades) tienen lo que se llama modificadores y hay básicamente 2 tipos:

- **Modificadores de acceso** (Visibilidad desde otras clases):
  - **pública** (accesible para todo el mundo),
  - **protegida** (accesible para el elemento que lo declara y heredados) y
  - **privada** (accesible para el elemento declarante).
- **Modificadores de no acceso**:
  - **static:** define miembros que pertenecen a la clase en sí, no a instancias particulares de clase.

POO

- **Modificadores de no acceso**:
  - **final:** no se pueden modificar.
    - **clases**, no se pueden heredar,
    - **métodos**, no se pueden sobreescribir,
    - **variables**, no se pueden modificar.

- Un **fichero java** sólo puede contener **una clase pública**.
- Además, **el nombre del fichero debe coincidir con el nombre de la clase pública**. Si nuestro fichero se llama **miFichero. java** la clase pública declarada en el mismo tendría que ser: **public class miFichero { ... } .**
- Se pueden declarar más clases dentro del fichero, pero para uso interno de la clase principal: class ClaseInterna { ... }

Ejercicio Taller Mecánico

Enunciado:

Tenemos un taller de reparación de coches y queremos realizar un programa con el que gestionar los coches que entran, salen y están en el taller.

Para ello, vamos a crear una clase Coche. Los datos de cada coche que queremos registrar son: propietario, matrícula, marca y si está reparado.

Las opciones que se mostrarán por pantalla al entrar en el programa son:

1.- Entrada nueva reparación: Se registran todos los datos del coche que se recepciona. Se recepciona como reparado = false.

2.- Salida de coche reparado: Se busca el coche en el taller por el nombre de propietario. Localizado el coche, se cambia a reparado = true y se entrega al propietario, saliendo del taller.

3.- Listado de vehículos: Se listan todos los vehículos en el taller, mostrando todos los parámetros.

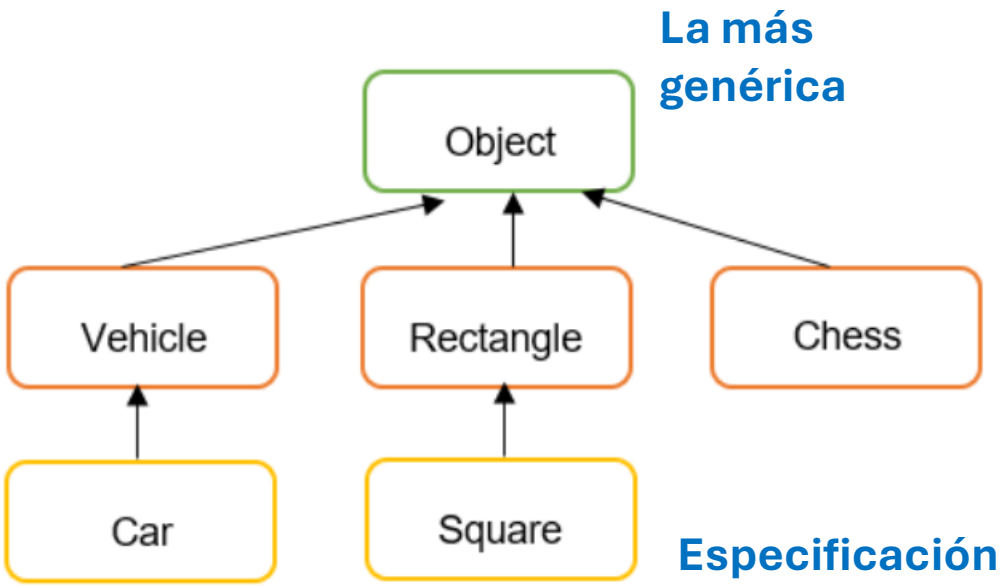
4.- Salir: salir del programa.

Herencia y polimorfismo

En Java, las clases se organizan en una **jerarquía de clases**.

En lo más alto de la jerarquía se encuentra la clase más general, **en Java, la clase Object**.

Las clases debajo de Object en la jerarquía se conocen como sus **subclases**.



Las clase **Object** (**la más genérica**) tiene características comunes a todos los objetos de Java.

Si descendemos en la jerarquía, las clases son más **específicas.**

Ej: Un rectángulo es un objeto: con atributos específicos (largo y ancho), comunes a todos los rectángulos. Rectangle es una **subclase** de Object y Square es una **subclase** de Object y Rectangle

Ej: Vehicle es un objeto: pero no tiene los atributos específicos de los rectángulos.

Herencia y polimorfismo

En Java, se usa la palabra clave **extends** para indicar que una clase B hereda su comportamiento de la clase A.

Para invocar a un método de la clase padre, se utiliza la palabra clave **super**.

Polimorfismo: una clase padre puede definir un comportamiento general y las clases hijas lo pueden implementar o modificar de diferentes maneras.

Cualquier método de una clase padre puede ser sobrecargado/sobreescrito por sus clases hijas. (Ej **toString()** de Object)

```
class A {  1 usage
    public void miMetodo() {    };
}
class B extends A{  no usages
    public void miMetodo() {  1 usage
        super.miMetodo();

    };
}
```

Herencia y polimorfismo

A diferencia de C++, **Java NO admite la herencia múltiple**, es decir, una clase Java sólo puede heredar de una clase padre, y no de más de una.

El motivo es para para evitar problemas de **ambigüedad y complejidad**.

¿Qué sucede cuando dos padres tienen diferentes versiones del mismo método?

Para sortear esta limitación, en Java se dispone de **clases abstractas e Interfaces**.

```
class ClaseA {  1 usage
    public void saludar() {  no usages
        System.out.println("Hola desde Clase A");
    }
}
class ClaseB {  1 usage
    public void saludar() {  no usages
        System.out.println("Hola desde Clase B");
    }
}
// En este caso Java no permite esto
class ClaseC extends ClaseA, ClaseB {  no usages
    // ¿Cuál método de saludar se hereda aquí?
}
```

Ejercicio 6

Ejemplo a realizar en clase:

Crear una clase Persona con los atributos nif y nombre (con getters, setters y constructor con parámetros).

Crear la clase Alumno como derivada de la clase Persona. La clase Alumno contendrá solamente el atributo curso, el nombre y el nif son los heredados de Persona.

Crear un **main** en el cual se cree un objeto de la clase Alumno y se guarden y obtengan datos para sus 3 atributos.

Anadir a la clase Persona un método leer() para introducir por teclado los valores a los atributos de la clase.

Modificar la clase Alumno para poder leer también el curso del alumno que se

introduce.

Ejercicio 6-b

Se quiere gestionar las personas vinculadas con el IES, que se pueden clasificar en: estudiantes, profesores y personal de servicio.

Por cada persona se debe conocer su nombre y apellidos, su DNI y su estado civil.

Si son empleados del IES se debe saber su año de incorporación y el número de despacho que tienen asignado.

Los profesores sólo pueden pertenecer a un departamento determinado y el personal de servicio a una sección concreta.

Por último, se tiene que almacenar a qué curso pertenecen los estudiantes.

Prueba a crear varios objetos de cada una de las clases propuestas y a mostrar la información de cada uno de ellos (puedes redefinir el método toString para ver la información de cada objeto de una forma concreta).

Abstracción

La abstracción es un proceso para **ocultar los detalles de implementación** y mostrar solo la **funcionalidad** al usuario.

Las clases abstractas pueden **contener métodos abstractos y métodos no abstractos**, que serán heredados por los hijos.

Un método abstracto no tiene cuerpo. (No tiene código). **Solo se escribe la signatura del método con la palabra reservada abstract**. Y **será la clase hija la que tiene que implementar el código del método abstract**o.

Veamos como ejemplo clase abstracta Animal:

```
public abstract class Animal {

    protected String nombre;

    public Animal(String nombre) {
        this.nombre = nombre;
    }

    public abstract void alimentar();
    public abstract void mover();

    public String getNombre() {
        return nombre;
    }
}
```

Abstracción

La clase hija **no abstracta** hereda el método o los métodos abstractos del padre y **debe desarrollarlos como métodos no abstractos**, es decir, con código.

No todo lo definido en una clase abstracta debe ser abstracto. Sin embargo, **si una clase contiene un método abstracto, entonces la clase en sí debe declararse abstracta**.

```
public class Dog extends Animal {

    public Dog(String nombre) {
        super( nombre: nombre);
    }

    @Override
    public void alimentar() {
        System.out.println("El perro " + getNombre() + " está comiendo.");
    }

    @Override
    public void mover() {
        System.out.println("Se está moviendo");
    }
}

public static void main(String[] args) {
    Dog d = new Dog( nombre: "Max");
    d.alimentar();
```

Clases abstractas

Ejemplo: Implementación de una clase **Polígono** que podrá definir propiedades comunes a varios polígonos (Circulo, cuadrado, …)

- Esta clase podría tener los atributos PosX, PosY, constructores y el método mover(), comunes para todas las subclases.
- A partir de esta clase se podría ir creando **subclases** que heredarán esos atributos y método comunes. Clase **Cuadrado**, clase **Círculo**, … además de algún atributo propio que gestionarán ya en sus propios constructores.
- Si queremos añadir **los métodos** para el cálculo del **área y del perímetro**, lo lógico sería añadirlos **en la clase base porque todos los polígonos podrán realizar el cálculo**.
- Pero…. OJO porque este cálculo no es el mismo para todos los polígonos.
- Para ello **creamos la clase Polígono como abstracta en la que se define un comportamiento, que posteriormente se implementará en cada subclase.**

Interfaces

Una interfaz es una clase que define un método común a varias clases, pero no lo implementa, estos métodos se deben implementar en las clases que hagan referencia a dicha interfaz.

Se podría considerar como **una clase abstracta pura:** todos sus métodos son abstractos y si tiene atributos son todos constantes.

Hay principalmente **tres razones para usar la interfaz**.

- Para lograr la abstracción.
- Dan algunas ventajas de herencia múltiple, sin las desventajas de la herencia.
- Para obtener un mayor desacoplamiento del código.

Las interfaces son muy usadas, de hecho, muchas de las librerías de Java hacen un uso extensivo de las interfaces.

Interfaces

Una clase hereda de un solo padre, pero puede implementar a la vez una o varias interfaces.

Por ejemplo, una clase **Coche** podría extender de la clase **Vehiculo**. **La herencia le da todos los métodos y variables de instancia.**

Pero si **Coche** también **implementa la interfaz Impuestos**, entonces su definición **debe contener código para TODOS los métodos enumerados en Impuestos**.

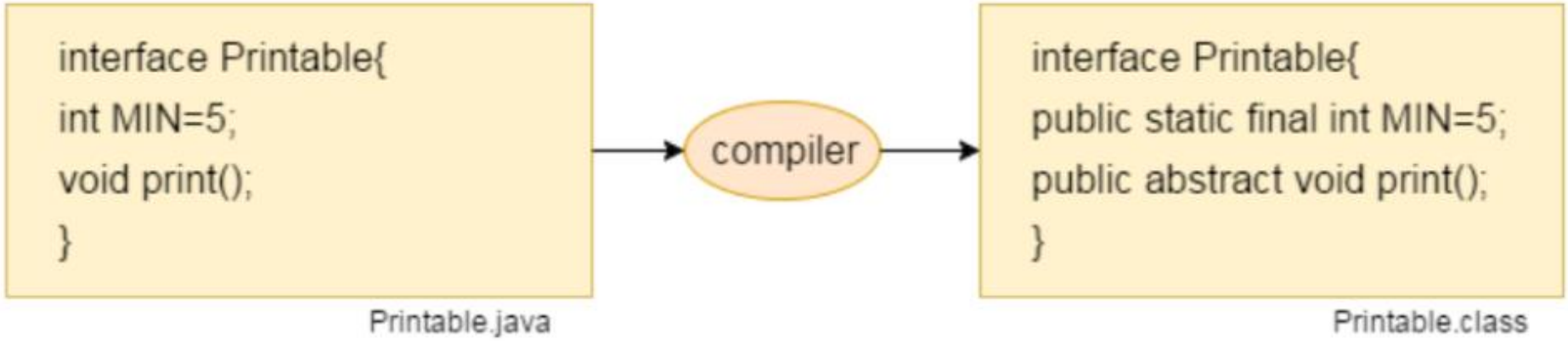
Al crear una interfaz en java, el compilador agrega las palabras clave, es decir:

- **public abstract** cuando se define un método,
- **public static final** en el caso de las.

por lo que se puede omitir en los encabezados de los métodos.

Interfaces

Ejemplo:



Las interfaces son **por defecto abstractas y públicas, no siendo necesario hacer esta declaración de manera explícita.**

(Ver ejemplo de código: Impuesto, TelefonoMovil, LineaDatos

Arrancable, Vehículo, Servicio)

Ejercicio Interfaces

Ejercicio: Crear una aplicación en la que dadas varias entidades o productos, se les aplique el iva de forma distinta a cada uno. Para ello:

- Crea una interfaz llamada “Impuesto” que defina un método CalcularImpuesto()
- Crea una clase “Producto” con los atributos precio y porcentaje de iva, en la que el cálculo del impuesto sería el precio por el porcentaje de iva.
- Crea una clase “Salario” con los atributos salario bruto y porcentaje de irpf, en la que el cálculo del impuesto sería el salario bruto por el porcentaje de irpf.
- Crea una clase “Propiedad” con los atributos valor catastral y porcentaje de ibi, en la que el cálculo del impuesto sería el valor catastral por el porcentaje de ibi.
- Crea una clase principal que muestre cómo se pueden utilizar estas implementaciones para calcular diferentes tipos de impuestos según el tipo de entidad.

Interfaces

Desde Java 8 podemos incluir en una interfaz:

- **Métodos con cuerpo o implementación**: se denominan ***default methods***.
  - **Se heredan como cualquier método ordinario** más
  - **Se pueden sobreescribir en la clase que lo implementa en la interfaz que lo hereda**.

Entonces… ¿Qué ocurre si una misma clase implementa dos interfaces con el mismo método por Defecto? ¿Cuál de los métodos se ejecutaría?

En estos casos, **Java obliga a que la clase que hereda el mismo método por defecto implementado en las dos interfaces sea sobreescrito**.

Interfaces

Desde Java 8 también se puede incluir en una interfaz:

- **Métodos estáticos con cuerpo o implementación**.
  - **Pertenecen a la interfaz y no a las instancias de las clases que la implementan**.
  - **No se pueden sobreescribir o cambiar** en ninguna clase que lo implemente**.**
  - Se invocan directamente desde la interfaz, no desde las instancias de clase

Interfaces

¿Por qué el uso de **default o static methods** en el interior de una interfaz?

Ejemplo: tenemos una interfaz con muchos métodos implementados en muchas clases, ¿qué pasa si añadimos un nuevo método en la interfaz?

Todas las clases que implementen esa interfaz se verán afectadas con errores, hasta que implementen ese nuevo método.

Aunque no es una tarea complicada puede llegar a ser tediosa(búsqueda en todo el código del proyecto en donde se utiliza la interfaz, cambio y pruebas)

O incluso puede que no sepamos todavía como implementarlo en todas las clases.

Ejemplo.

Interfaces funcionales

Son aquellas que contienen un único método o comportamiento.

Se añade al principio de la declaración inicial de la interface: **@FunctionalInterface**

Ejemplo:

```
3    @FunctionalInterface  no usages
4    public interface IFOperacion {
5        double calcular(int a, int b);  no usages
6    }
```

¿Tendríamos que implementar este método en las clases que lo implementan? Sí:

```
3    public class Operacion implements IFOperacion {
4
5        @Override  no usages
6        public double calcular(int a, int b) {
7            return a+b;
8        }
9    }
```

Clases anónimas

Se crean **a partir de una interfaz funcional**.
Se puede instanciar definiendo el método abstracto a implementar en la propia inicialización sin necesidad previa de definir una clase.

```
IFOperacion opCA = new IFOperacion() {
    public double calcular(int a, int b) {
        int c=a+b-10+32;
        return c;
    }
};
```

//).

Funciones lambda

Se puede incluso omitir la definición de la clase abstracta.

- Se indica el conjunto de parámetros necesarios (izquierda de la flecha)
- y el código que define el método abstracto de la Interfaz funcional (derecha de la flecha)

```
IFOperacion op2 = (a, b) -> {
    System.out.println("Cálculo en una expresión lambda");
    int c =(a+b)*10;
    return c;
};
```

Ejercicios - Operaciones con Clases anónimas y lambdas

Ejercicio 7:

Crea una interfaz funcional llamada OperacionMatematica que tenga un método double calcular(double a, double b).

Crea un programa en donde uses clases anónimas y lambdas para realizar las operaciones de suma, resta, multiplicación y división. Muestra los resultados que vas obteniendo por pantalla.

Ejercicio 8:

Crea una interfaz funcional llamada ConcatenarCadenas que tenga un método abstracto String concatenar(String a, String b).

Usa una clase anónima y una expresión lambda para implementar la interfaz de modo que concatene dos cadenas de texto.

Llama al método con dos cadenas cualesquiera y muestra el resultado.

Excepciones

Una **excepción** es un **evento que interrumpe el flujo normal del programa** cuando ocurre un error.

Puede **provocar que el programa no funcione bien o termine de forma inesperada.**

Es un mecanismo de Java **para manejar situaciones inesperadas que pueden ocurrir durante la ejecución de un programa.**

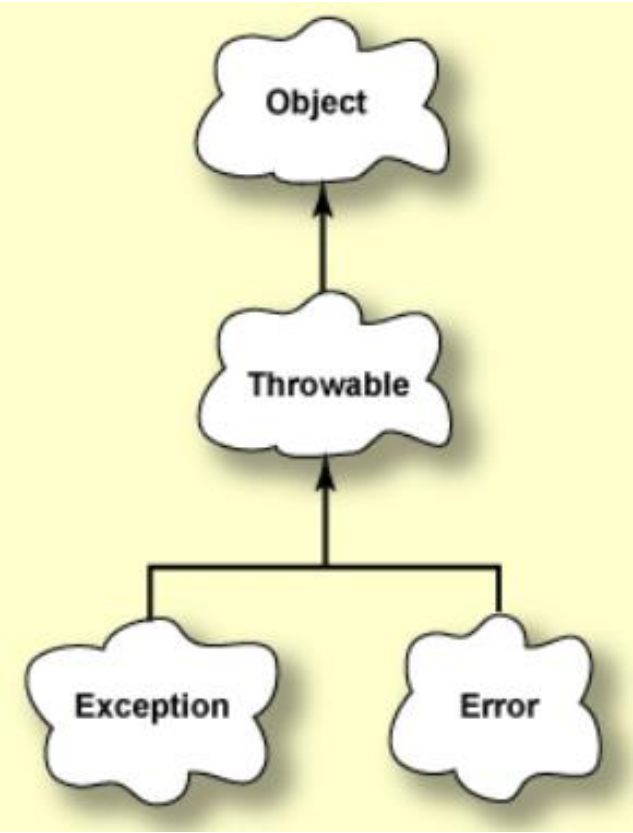
Cuando se produce una excepción, se **lanza** (throw) un objeto que representa ese error.

Excepciones

Exception y **Error** heredan **Throwable**. Es por ello que **cuando ocurre uno de estos problemas en código se puede "lanzar“** un objeto de clase Throwable **(throws)** .

Se pueden capturar o recuperar excepciones, pero no se pueden capturar errores (situaciones críticas del sistema. Irreparables desde el punto de vista del programa).

Las excepciones pueden ser detectadas por una parte del programa que intenta controlar el problema.



Ejemplos de situaciones que provocan una excepción:

- No hay memoria disponible para asignar
- Acceso a un elemento de un array fuera de rango
- Leer por teclado un dato de un tipo distinto al esperado
- Error al abrir un fichero
- División por cero
- Problemas de Hardware

Excepciones

Si la excepción no se trata, el manejador de excepciones realiza lo siguiente:

- Muestra la descripción de la excepción.
- Muestra la traza de la pila de llamadas.
- Provoca el final del programa.

Tipos de excepciones

- **Checked exceptions** (Excepciones verificadas).

  Son aquellas que **se comprueban en tiempo de COMPILACIÓN**, es decir, un método debe hacer algo al respecto sino el programa no compila. Deben manejarse con:

  - Bloques **try-catch**

  - O Elevar la excepción al nivel superior mediante la sentencia **throws** en la declaración del método.

  Ejemplos: IOException, SQLException.

Tipos de excepciones

- **Checked exceptions** (Excepciones verificadas)

Código con excepción verificada:

```
DataOutputStream fos = new DataOutputStream(
        new FileOutputStream( name: "datos.dat"));
fos.writeInt( v: 0);
fos.close();
```

Error al compilar, 2 soluciones:

- **Se eleva al método que la invoca**
- **Se captura con un try-catch**

```
public static void main(String[] args) throws IOException {
    DataOutputStream fos = new DataOutputStream(
            new FileOutputStream( name: "datos.dat"));
    fos.writeInt( v: 0);
    fos.close();
```

```
try {
    DataOutputStream fos = new DataOutputStream(
            new FileOutputStream( name: "datos.dat"));
    fos.writeInt( v: 0);
    fos.close();
} catch (FileNotFoundException e) { //todas pueden de
    System.out.println(e.getMessage());
} catch (IOException e) {
    System.out.println(e.getMessage());
}
```

Tipos de excepciones

- **Unchecked exceptions** (Excepciones no verificadas).

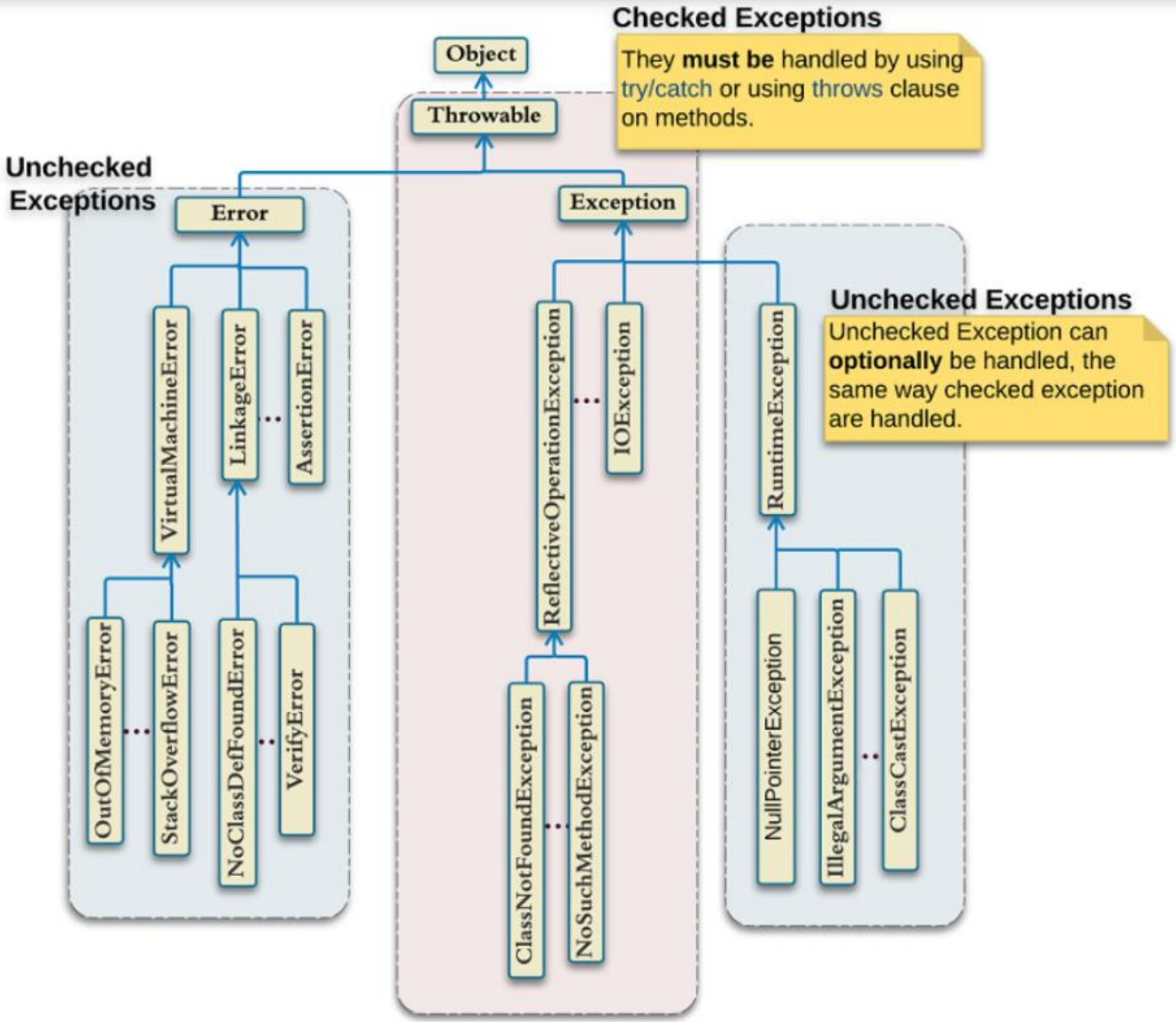
Ocurren en tiempo de EJECUCIÓN, es decir, no se verifican por el compilador.

Depende de los programadores especificar o detectar las excepciones unchecked.

Ejemplo: el siguiente programa compila sin errores:

```
public static void main(String[] args) {
        int x = 0;
        int y = 10;
        int z = y / x;
}
```

Compila sin error porque la división por 0 lanza una excepción no verificada (), pero al ejecutar sí se lanza excepción, con lo cual nosotros tendríamos que darnos cuenta y controlar la excepción.


Checked Exceptions
They must be handled by using try/catch or using throws clause on methods.
Object
Throwable
Unchecked Exceptions
Error
Exception
VirtualMachineError
LinkageError
AssertionError
OutOfMemoryError
StackOverflowError
NoClassDefFoundError
VerifyError
ReflectiveOperationException
IOException
ClassNotFoundException
NoSuchMethodException
Unchecked Exceptions
Unchecked Exception can optionally be handled, the same way checked exception are handled.
RuntimeException
NullPointerException
IllegalArgumentException
ClassCastException

Sintaxis de excepciones.
esquema general para tratar excepciones:

```
try{
    //Instrucciones que se intentan ejecutar
    //Si se produce una situación inesperada se lanza una excepción
}
catch(tipoExcepcion e){                    //manejador de excepciones
    //Instrucciones para tratar esta excepción
}
catch(otroTipoExcepcion e){      //Se pueden escribir tantos bloques catch como sean necesarios

    //Instrucciones para tratar esta otra excepción
}
finally{
    // instrucciones que se ejecutarán siempre después de un bloque try - catch
}
```

Ejemplo (0):

```
Scanner sc = new Scanner(System.in);
int numero;
System.out.println("Introduzca un entero:");
try {
   numero = sc.nextInt();
}
catch (InputMismatchException e) {
   System.out.println("No has introducido un número.");
   e.printStackTrace();
}
System.out.println("Sigue ejecutándose el programa");
```

Cuando se introduce por teclado algo distinto a un entero:

Se lanza dentro del bloque try una excepción del tipo **InputMismatchException**,

el bloque catch comienza a ejecutarse inmediatamente.

Se omiten las sentencias restantes en el bloque try y se continúa ejecutando el código después del tratamiento de la excepción.

Ejemplos de excepciones

```java
public static void main(String[] args) {
    int resultado = 10 / 0; //ArithmeticException
    System.out.println("Resultado: " + resultado);
}
```

```
Exception in thread "main" java.lang.ArithmeticException Create breakpoint : / by zero
    at Tema1.Excepciones.Ejemplo1.main(Ejemplo1.java:6)
```

Si la controlamos con try-catch:

```java
public static void main(String[] args) {
    try {
        int resultado = 10 / 0; // ArithmeticException
        System.out.println("Resultado: " + resultado);
    } catch (ArithmeticException e) {
        System.out.println("Error: División por cero no permitida.");
    }
}
```

```
Error: División por cero no permitida.
```

Ejemplos de excepciones

```java
public class Ejemplo2 {
    public static void main(String[] args) {
        try {
            int[] numeros = {1, 2, 3};
            System.out.println(numeros[5]); // ArrayIndexOutOfBoundsException
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: Índice fuera de los límites del array.");
        } finally {
            System.out.println("Este bloque se ejecuta siempre.");
        }
    }
}
```



```
Error: Índice fuera de los límites del array.
Este bloque se ejecuta siempre.
```

Ejercicios de excepciones

Ejercicio 9: Crea un programa en el que se pida por teclado un número y se inserte una letra ¿Qué ocurre? ¿Qué excepción se lanza? Modifica el programa para que se capture la excepción, el programa no falle y pueda seguir en funcionamiento. El programa no debe parar hasta que se introduzca un número.

Ejercicio 10: Crea un programa en el que tengamos un array con X elementos. El programa solicitará que se introduzca un número para obtener el elemento en la posición que ocupa dicho número y posteriormente se mostrará el elemento cuya posición ocupa el número introducido. En primer lugar, introducir una letra en vez de un número y en segundo lugar introducir un número superior al total de elementos que tiene el array. Averiguar qué excepciones se lanzan para, posteriormente tratarlas, capturarlas y mostrar mensajes en el bloque finally y al salir del try.

Ejercicio 11: Escribe un programa que lea por teclado un número x. Con un bloque try / catch que arroje una excepción si el valor de la variable x es menor que cero. La excepción debe ser una instancia de Exception y, cuando se detecta, el mensaje devuelto por getMessage() debe ser "ERROR: valor negativo en la coordenada X". También se debe comprobar que el número introducido sea un número, de lo contrario, capturar el error para que el flujo del programa no salga de forma abrupta. Al final, en el bloque “finally” indicar si se ha podido comprobar la variable o el flujo ha salido por no leer un número.

Ejercicio 12: Escribe un programa que muestre un menú iterativamente con opciones para dar elegir al usuario. Maneja con un try catch si el usuario inserta una opción que no es numérica, es decir letras, mostrando el mensaje: “Sólo se permiten números”. Si el usuario inserta una opción correcta acaba el programa con el mensaje: “Opción válida”, si la opción no está entre las ofrecidas vuelve a mostrar el menú.

Referencias

- Documentación oficial de Java 16:
  - https://docs.oracle.com/en/java/javase/16/docs/api/index.html
- Otros tutoriales de Java:
  - https://www.javatpoint.com/java-tutorial
  - https://www.tutorialspoint.com/java/index.htm
- JavaDoc:
  - https://www.oracle.com/at/technical-resources/articles/java/javadoc-tool.html

Referencias

- Introducción a IntelliJ IDEA (JetBrains):
  - https://www.jetbrains.com/help/idea/discover-intellij-idea.html
- Expresiones regulares:
  - https://regex101.com/
- Code Snippets en IntelliJ IDEA:
  - https://www.jetbrains.com/help/idea/using-live-templates.html
- Atajos de teclado en IntelliJ IDEA:
  - https://www.jetbrains.com/help/idea/mastering-keyboard-shortcuts.html
  - https://www.jetbrains.com/help/idea/configuring-keyboard-and-mouse-shortcuts.html